Journal of Applied Technology and Informatics
Volume 1, Number 3, Tahun 2021, ISSN 2807-3347
Journal Homepage: https://jurnal.del.ac.id/index.php/jati

PENGUJIAN API WEBSITE UNTUK PERBAIKAN PERFORMANSI APLIKASI DITENUN

Arlinta Christy Barus[1*], Johannes Harungguan[2*], Efren Manulu[3*]

[1,2,3]Institut Teknologi Del, Sumatera Utara, Indonesia

*Penulis Korespondensi:
[1]arlinta@del.ac.id, [2]Johannes@del.ac.id, efrenmanalu1605@gmail.com

Received 1 Oktober 2021; Revised 2 November 2021; Accepted 21 November 2021

ABSTRAK

Aplikasi DiTenun merupakan piranti cerdas penghasil motif tenun nusantara yang dibangun pada tahun 2016 yang berjudul "Piranti Cerdas Penghasil Motif Tenun Nusantara". Melihat adanya permasalahan pada aplikasi DiTenun dimana performansi menjadi kurang baik pada saat jumlah *user* meningkat maka dilakukan penelitian untuk melakukan pengujian performansi dengan tingkat uji beban di sejumlah *virtual user*. Adapun *server* yang digunakan pada aplikasi DiTenun merupakan *server* dari Institut Teknologi DEL, sehingga pengembangan aplikasi DiTenun berasal dari *server* IT DEL. Untuk sekarang ini Aplikasi DiTenun menggunakan penerapan API sebagai sarana dan proses pembangunan serta pengembangan yang dilakukan oleh *developer*nya. Oleh sebab itu pada penelitian ini akan dilakukan pengujian API pada aplikasi DiTenun dengan mengukur tingkat performansi menggunakan pengujian *Performance test* dengan menggunakan tools JMeter dan mengukur tingkat uji beban di sejumlah *virtual user* yang melakukan interaksi pada sistem secara bersamaan menggunakan pengujian *Load Test*. Peneliti juga akan melakukan pengukuran *baseline web service* aplikasi DiTenun dari *response time* API berdasarkan beban data yang dipanggil. Setelah dilakukan analisis, berdasarkan pengujian performansi yang dilakukan pada API dengan menggunakan metode *Load Test* diperoleh bahwa performa oleh sebagian besar API mengalami penurunan seiring dengan bertambahnya jumlah *user* yang mengakses, meskipun tidak ditemukannya *error* pada pengujian yang dilakukan. Adapun pada hasil pengukuran *baseline web service*, peneliti memperoleh hasil *response time* yang dapat dijadikan acuan sebuah API yang akan dijalankan dalam sebuah sistem pada *server* yang telah dipakai oleh peneliti. Hasil dari penelitian ini dapat digunakan oleh pengelola DiTenun untuk melakukan perbaikan performansi DiTenun.

Kata Kunci: *website* DiTenun, API, Pengujian Performansi, *Baseline Response time*, *Load Test*

ABSTRACT

The DiTenun application is an intelligent device for producing Indonesian weaving motifs which was built in 2016 entitled "Intelligent Device for Producing Indonesian Weaving Motifs". Seeing the existence of problems in the DiTenun application where performance becomes less good when the number of users increases, it is carried out to perform performance testing with Load Test levels in virtual user numbers. The server used in the DiTenun application is a server from the DEL Institute of Technology, so that the development of the DiTenun application comes from the IT DEL server. For now, the DiTenun application uses the application of the API as a means and process of development and development carried out by the developer. Therefore, in this study, API testing will be carried out on the DiTenun application by measuring the level of performance usingJMeter to do the Performance test and measuring the level of the number of virtual users interacting with the system simultaneously using the Load Test. The researcher will also measure the baseline of the DiTenun application web service from the API response time based on the data load being called. After the analysis, based on the performance testing performed on the API using the Load Testing method, it was found that the performance of most of the APIs decreased with the increase in the number of users accessing, although no errors were found in the tests carried out. As for the results of the baseline web service measurement, the researcher obtained the results of the response time which can be used as a reference for an API that will be run in a system on the server that has been used by the researcher. The result could be used by DiTenun to improve the performance of DiTenun.

Keyword: DiTenun Website, API, Performance testing, Baseline Response time, Load Test

I. PENDAHULUAN

Pengujian perangkat lunak atau *software testing* merupakan tahap yang dilakukan setelah tahap implementasi dalam proses pembangunan perangkat lunak. Menurut Oscar dalam (MZ, 2016), pengujian diperlukan untuk menghasilkan sebuah produk yang memiliki nilai, produktivitas dan kualitas yang tinggi, di mana kualitas aplikasi itu sendiri bergantung kepada kepuasan pelanggan atau pengguna (Cholifah, et al., 2018). Dengan adanya pengujian perangkat lunak, maka pengalaman *user* menjadi baik dan menyenangkan dalam berinteraksi pada suatu sistem sehingga dapat menarik pelanggan potensial untuk menggunakan aplikasi tersebut.

Salah satu faktor terpenting dalam menentukan kualitas suatu perangkat lunak adalah performansi dari aplikasi atau sistem tersebut. Performansi yang dimaksud mengacu pada kemampuan sistem untuk memberikan respon terhadap transaksi informasi dengan lebih baik, cepat, dan akurat (Patel & Gulati, 2015). Sehingga hasil dari pengukuran performansi tersebut dapat diperoleh informasi apakah sistem pada aplikasi sudah berperilaku memuaskan atau tidak sesuai dengan kebutuhan *user*.

Adapun pengujian *Load Test* merupakan salah satu jenis pengujian performansi yang memeriksa bagaimana sistem berfungsi dengan banyaknya *virtual user* secara bersamaan melakukan interaksi pada sistem selama periode waktu tertentu. Dengan kata lain, pengujian *Load Test* dilakukan untuk mengukur bagaimana sistem menangani volume beban berat. Salah satu *tools* dalam pengujian *open- source Load Testing* adalah Jmeter.

API atau *web service* merupakan salah satu faktor yang mempengaruhi performansi suatu sistem. API *(Application Programming Interface)* merupakan sekumpulan prosedur yang dapat digunakan oleh aplikasi lain untuk memenuhi kebutuhannya. API sendiri dapat diartikan sebagai antarmuka yang dibangun oleh pengembang sistem agar sebagian atau keseluruhan fungsi sistem dapat diakses secara terprogram (Putra, 2018). API dibangun dengan tujuan mampu mempersingkat proses pengembangan sebuah perangkat lunak agar fitur-fitur yang sama bisa digunakan oleh perangkat lunak lain dan pengembang tidak perlu membangun fitur yang sama lagi. API sendiri merupakan sebuah *class* yang disusun untuk dapat menghubungkan basis data dengan perangkat lunak. Melalui *class* ini nantinya akan dijadikan sarana untuk mengakses dan mengeksekusi berbagai perintah pada perangkat lunak. *Class* yang terdapat pada *web service* memiliki fungsi dan peran tersendiri sehingga setiap class akan diuji kemampuannya dalam memproses perintah/*request*. *Web service* berperan sebagai komponen yang menghubungkan antarmuka dengan *server* yang mensupport prosedur dan fungsi yang akan memproses *request* dan *response*.

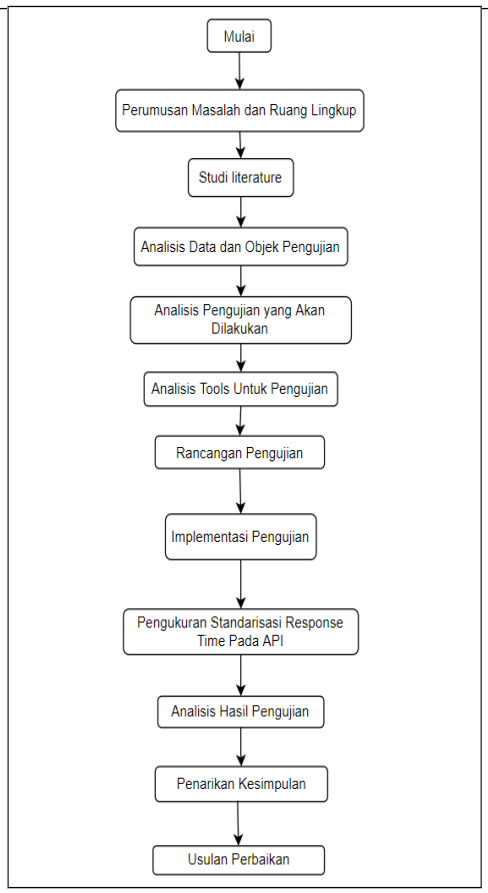
Sejumlah penelitian yang telah membahas mengenai pengujian terhadap API antara lain adalah penelitian yang berjudul "Performance Evaluation of Restful Web service for Mobile Device". Pada penelitian ini membahas mengenai bagaimana kinerja pada Restful API yang akan dibandingkan dengan kinerja web service konvensional (SOAP) untuk menemukan hasil kinerja yang terbaik yang sesuai diterapkan pada mobile device (Hatem Hamad, 2009). Penelitian lain yang berjudul "A Web service Trust Evaluation Model Based-on Small-World Network" yang membahas dan fokus terhadap security saja (Fengming Lui 2013). Penelitian selanjutnya adalah "A Performance Evaluation Method and It's Implementation for Web service" dimana penelitian ini membahas mengenai bagaimana metode evaluasi pada web service menggunakan "web service evaluation system" yang menjadi alat ukur untuk sejumlah parameter delay yang berbeda dan sudah ditetapkan oleh peneliti sebelumnya (Fei Sha, 2014).

Sebagai perangkat lunak yang menjadi objek penelitian pada Tugas Akhir ini adalah aplikasi DiTenun yang dikembangkan oleh para peneliti dosen Institut Teknologi Del dengan berjudul "Piranti Cerdas Penghasil Motif Tenun Nusantara". Melihat adanya permasalahan pada aplikasi DiTenun dimana performansi menjadi kurang baik pada saat jumlah user meningkat. Sehingga dilakukan penelitian untuk melakukan pengujian performansi dengan tingkat uji beban di sejumlah virtual user. Adapun server yang digunakan pada aplikasi DiTenun merupakan server dari Institut Teknologi DEL, sehingga pengembangan aplikasi DiTenun berasal dari server IT DEL. Dengan memperhatikan performansi pada server IT DEL maka dapat diperoleh pengukuran performa server pada aplikasi DiTenun dalam menjalankan kebutuhan user. Untuk sekarang ini Aplikasi Ditenun menggunakan penerapan API sebagai sarana dan proses pembangunan serta pengembangan yang dilakukan oleh developernya. Oleh sebab itu, penelitian ini melakukan pengujian API pada aplikasi DiTenun dengan mengukur tingkat performansi menggunakan Performance test dan mengukur tingkat uji beban di sejumlah virtual user dengan interaksi pada sistem secara bersamaan menggunakan pengujian Load Test. Pengujian performansi dan pengujian Load Testing tersebut dilakukan untuk memperoleh hasil pengukuran pengujian pada kondisi jika semakin banyak pengunjung/user yang menggunakan aplikasi DiTenun, maka bagaimana performansi yang dilakukan oleh server untuk memberikan kebutuhan user.

Pengujian terhadap API yang ada pada Aplikasi Ditenun dilakukan pada API website aplikasi DiTenun karena selama proses pengembangan, belum pernah dilakukannya analisis pengujian API pada aplikasi DiTenun yang telah dikembangkan oleh peneliti sebelumnya. Penelitian terkait pengujian performansi ini dirasa penting karena diperlukan pengujian terhadap service apakah sudah sudah memiliki performa yang baik atau masih ditemukannya kesalahan pada waktu eksekusi atau proses request-response atau kesalahan- kesalahan lainnya. Peneliti akan melakukan analisis hasil pengujian performansi terhadap API website DiTenun yang nantinya dapat menjelaskan kinerja API DiTenun yang dapat digunakan sebagai referensi perbaikan dan penyempurnaan dalam mengembangkan.

II. METODE PENELITIAN

Proses penelitian yang dilakukan oleh peneliti mencakup, Rumusan Masalah dan Ruang Lingkup, Studi Literatur, Analisis Data dan Objek Pengujian, Analisis Aspek Pengujian yang Akan Dilakukan, Analisis *Tools* Untuk Pengujian, Rancangan Pengujian, Proses Pengujian, Pengukuran *Baseline Response time* Pada API, Analisis Hasil Pengujian, Penarikan Kesimpulan, Usulan Perbaikan. Tahapan penelitian ditunjukkan pada Gambar 1.



Gambar 1. Desain Eksperimen

Tahapan-tahapan pada Gambar 1 dijalankan sebagai berikut.

1. Tahap pertama adalah perumusan masalah. Tahapan pertama yang dilakukan yaitu rumusan masalah dan ruang lingkup. Tahapan awal yang dilakukan dalam melakukan penelitian ini adalah menemukan Rumusan Masalah berdasarkan latar belakang yang disusun. Adapun rumusan masalah yang didapatkan adalah bagaimana cara menerapkan pengujian performansi terhadap API *Website* DiTenun dan bagaimana hasil pengukuran *baseline response time* pada API *Website* DiTenun .Setelah diperoleh rumusan masalah, kemudian ditentukan ruang lingkup penelitian agar diketahui batasan dan fokus penelitian yang akan dilakukan.
2. Tahap kedua yang dilakukan yaitu studi literatur dimana yang dilakukan adalah melakukan studi literatur mengenai API dan pengujian-pengujian yang sedang berkembang saat ini dalam menguji sistem suatu Aplikasi. Studi Literatur dapat dijadikan sebagai dasar dasar teori yang dapat berguna untuk penelitian yang akan dilakukan.
3. Tahapan Ketiga yang dilakukan yaitu analisis data dan objek pengujian. Pada Tahapan ini akan dilakukan Analisis terhadap Objek Pengujian yaitu Aplikasi Ditenun sendiri. Analisis yang dilakukan meliputi analisis *server* yang sedang dipakai, susunan database yang digunakan oleh Aplikasi Ditenun, Fungsi-fungsi yang dibangun, *web service* serta API yang akan diuji oleh peneliti.
4. Tahapan keempat yang dilakukan yaitu analisis aspek pengujian yang akan dilakukan. Selanjutnya akan dilakukan analisis terhadap pengujian yang akan dilakukan terhadap API itu sendiri. Pengujian yang akan dilakukan terhadap aplikasi DiTenun yaitu pengujian pengujian performansi dengan menggunakan metode *Load Test*. Metode- metode performansi tersebut diadaptasi dari metode pengujian performansi pada *Website*.
5. Tahapan kelima yang dilakukan yaitu analisis *tools* untuk pengujian. Setelah menemukan aspek- aspek yang dapat diuji dari suatu API, maka kemudian penulis akan melakukan Analisis serta penentuan *Tools* yang sesuai dengan Aspek serta spesifikasi dari API yang terdapat pada Aplikasi.
6. Tahapan keenam yang dilakukan yaitu rancangan pengujian. Setelah dilakukan analisis terhadap objek uji dan *tools* yang akan digunakan maka penulis akan melakukan pembuatan rancangan pengujian sesuai dengan Aspek serta metode- metode yang diterapkan dalam melakukan pengujian.
7. Tahapan ketujuh yang dilakukan yaitu proses pengujian. Tahapan ini merupakan tahapan implementasi pengujian. Pada tahap ini pengujian akan dilakukan secara berurutan dimulai dari pengujian Fungsionality kemudian ke pengujian Performansi dan ke Uji monitoring API.
8. Tahapan kedepalan yang dilakukan yaitu pengukuran *baseline response time* pada api. Pada tahap ini, peneliti melakukan penentukan dan analisis tolak ukur *response time* terhadap *web service* aplikasi *web* IT Del dengan melakukan analisis perhitungan terbentuknya *response time* pada *web service* tersebut.
9. Tahapan kesembilan yang dilakukan yaitu analisis hasil pengujian. Pada analisis hasil pengujian maka data-data yang akan dihasilkan dari hasil pengujian akan dianalisis serta akan diidentifikasi penyebab *error* atau penyebab rendahnya kinerja dari API tersebut.
10. Tahapan kesepuluh yang dilakukan yaitu Penarikan Kesimpulan. Dari hasil analisis pengujian maka akan ditarik kesimpulan mengenai Performansi dan *baseline response time* dari API.
11. Tahapan terakhir yang dilakukan yaitu usulan perbaikan. Usulan perbaikan didapatkan dengan cara menemukan penyebab *error* serta rendahnya kinerja dari API yang diuji dan kemudian akan ditusulkan solusi yang dapat diterapkan oleh *developer* kedepannya.

III. TINJAUAN PUSTAKA

3.1 Software Testing

Pengujian perangkat lunak atau *software testing* merupakan tahap yang dilakukan setelah tahap implementasi dalam proses pembangunan perangkat lunak. Pengujian Perangkat Lunak atau *software testing* juga dapat diartikan sebagai elemen kritis dari penjaminan kualitas dari suatu perangkat lunak serta merepresentasikan kajian pokok dari spesifikasi, desain dan pengkodean (Liana, 2015). Software sendiri merupakan suatu prosedur-prosedur atau aturan aturan yang dipakai oleh suatu mesin untuk memproses data-data yang telah diubah bentuknya ke dalam suatu program (Gunawan, 2010). Sedangkan yang dimaksud dengan *testing* menurut IEEE/ANSI dalam buku Edward Kit adalah proses dari suatu sistem operasi atau suatu komponen dalam kondisi

tertentu untuk diamati dan direkam hasilnya serta membuat evaluasi beberapa aspek dari komponen dan sistem tersebut. Pengujian perangkat lunak secara garis besar dibagi menjadi dua yaitu pengujian fungsional dan pengujian non-fungsional.

1) Pengujian Fungsional
   Pengujian fungsional merupakan pengujian yang paling umum digunakan karena merupakan pengujian yang dilakukan dengan menguji fungsional perangkat lunak apakah bekerja dengan baik. Pengujian ini berdasarkan kode sumber (*source code based testing*). Pengujian berdasarkan kode sumber ini sama dengan Black Box *Testing*.
2) Pengujian Non-Fungsional
   Pengujian Non-fungsional merupakan pengujian dengan melihat *internal* sistem. *Internal* sistem yang dimaksud perilaku spesifik sistem mencakup untuk memastikan operasi *internal* berjalan sesuai dengan spesifikasi, memeriksa bagaimana perangkat lunak bekerja dengan beban kerja yang diharapkan.

3.2 Performance Testing

Performance testing adalah jenis pengujian untuk memastikan perangkat lunak akan bekerja dengan baik dibawah beban kerja yang diharapkan (Permatasari, Ardani, Ma'ulfa, Ilhami, & Pratama, 2020). Untuk suatu kelayakan operasi, baik uji fungsional maupun uji non-fungsional harus dilakukan, sehingga menjamin bahwa perangkat lunak sudah siap. Hal ini menjadi suatu resiko yang signifikan khususnya pada sistem enterprais, dimana akan melayani pengguna dan transaksi dalam jumlah besar dengan intensitas dan reliabilitas yang memadai. Oleh karena itu, pada penelitian ini dikembangkan sebuah perangkat lunak uji yang dapat digunakan untuk melakukan uji kapasitas dan performansi. Fokus pada *performance test* sendiri terbagi atas beberapa bagian, yaitu:

- Speed – Untuk menentukan kecepatan respon dari aplikasi saat menerima *request* atau tindakan.
- Scalability – Untuk menentukan jumlah maksimum *user load* yang dapat ditangani oleh aplikasi.
- Stability – Untuk menentukan kestabilan aplikasi dalam berbagai beban.

3.3 Load Test

Load Testing adalah teknik *performance testing* yang mana respon sistem diukur dalam berbagai *load* condition. Pengujian ini membantu menentukan bagaimana software berperilaku ketika beberapa *user* mengakses software secara bersamaan (Permatasari, et al., 2020). Pengujian menggunakan tipe *Load Testing* ini juga memeriksa bagaimana sebuah sistem yang sedang dibangun atau dikembangkan tersebut bisa menangani masalah atau beban yang diujikan yang disesuaikan dengan keadaan nyata.

Load Testing dibutuhkan untuk melakukan simulasi terhadap akses aplikasi *website* secara simultan. Simulasi lebih baik dan tepat dilakukan dibandingkan melakukan pengujian dengan mengundang ratusan atau ribuan orang untuk mengakses aplikasi *website* tesebut secara bersamaan. *Load Testing* memiliki cara pengujian yang menguji berdasarkan keadaan yang sebenarnya pada dunia nyata agar *website*, aplikasi ataupun software yang sedang dibangun dan dikembangkan dapat dimanfaatkan dan bekerja secara maksimal ketika sudah digunakan oleh *user*.

3.4 Indikator Performance Testing

Indikator-indikator yang akan dianalisis dari hasil pengujian performa pada API Aplikasi Website Ditenun adalah sebagai berikut :

1) Response time
   Response time merupakan sebuah rentang waktu tunggu dimana client mengirimkan request ke server hingga server selesai melakukan response dari request tersebut. Secara teoritis *response time* akan semakin meningkat ketika jumlah user yang mengakses API secara bersamaan juga meningkat.
2) Throughput
   Throughput merupakan jumlah request user yang diproses server dalam satuan detik. *Throughput b*iasanya ditetapkan dengan *bits per second*. Namun pada pengujian performa dengan menggunakan Jmeter, *throughput* ditetapkan dalam *request per second (rps)*. Secara teoritis, *throughput* akan meningkat ketika beban yang ditawarkan meningkat, hingga maksimum dari kapasitas jaringan.
3) Error rate
   Error rate merupakan kalkulasi persentase transaksi yang gagal selama pengujian berlangsung. Berdasarkan metode pengujian load test, nilai *error rate* yang seharusnya dihasilkan adalah senilai 0%. Hal tersebut dikarenakan pengujian dilakukan dengan menggunakan jumlah user yang melakukan request secara bersamaan adalah jumlah yang normal (di bawah titik maksimum dari jumlah request secara bersamaan).
4) Concurrent User
   Concurrent user mengacu pada jumlah user yang mengakses API atau sistem dalam waktu yang bersamaan. Concurent user akan berpengaruh terhadap nilai *response time*, *throughput* dan *error.*

3.5 *Web* Service

Istilah *web service* menjelaskan cara standar untuk mengintegrasikan aplikasi berbasis *web*. *Web service* adalah media standar untuk menghubungkan komunikasi antara aplikasi klien dan *server*. *Web service* dirancang untuk melakukan tugas tertentu. Dalam *website* DiTenun menyediakan layanan *website* yang menyediakan tampilan untuk produk dan harga. Frontend menggunakan Bahasa pemrograman react-js yang memiliki kemampuan untuk berkomunikasi dengan layanan *web*. Contoh implementasi dari *Web service* adalah SOAP dan REST.

3.6 *Application Programming Interface*

Application Programming Interface (API) dapat diartikan sebagai antarmuka yang dibuat atau dikembangkan oleh pengembang sistem agar sebagian atau seluruh fungsi sistem dapat digunakan secara terprogram (Putra, 2018). API juga dapat diartikan sebagai *software interface* yang terdiri atas gabungan- gabungan perintah/instruksi yang disimpan di dalam bentuk *library* dan menjelaskan bagaimana agar suatu unit *software* dapat berinteraksi dengan unit *software* lainnya (Arianto, 2016). Penjelasan ini dapat dianalogikan dengan pembangunan sebuah rumah yang menyewa kontraktor. Pemilik rumah tersebut menyewa kontraktor dan menyerahkan segala urusan pembangunan rumah kepada kontraktor tersebut. Si pemilik rumah tidak akan tahu proses-proses yang akan dilakukan si kontraktor hingga bangunan tersebut pada akhirnya akan selesai. Dari penganalogian tersebut, bangunan yang dibangun dapat kita analogikan

sebagai *software* yang akan dibangun, dan kontraktor dianalogikan sebagai API yang mengerjakan bagian tertentu dari *software* tersebut tanpa harus diketahui bagaimana prosedur dalam melakukan pekerjaan tersebut (Reddy, 2011).

Di zaman sekarang ini, dimana zaman internet semakin berkembang, API dapat diimplementasikan pada bagian *server* dan dapat dipergunakan oleh beberapa system aplikasi yang dapat terhubung ke *server* dengan menggunakan protokol tertentu. Pada protokol HTTP, *Application Programming Interface* umumnya disebut sebagai *Web Application Programming Interface Server* atau *Web service*.

3.7 API *Testing*

Api testing merupakan salah satu tipe pengujian dalam *software testing* yang bertujuan untuk menguji API secara langsung. API *Testing* dapat dikatakan bagian dari *integration testing* untuk memastikan apakah system yang kita uji sudah sesuai dengan ekspektasi kita atau belum. Untuk saat ini, pengujian API lebih mudah dilakukan daripada pengujian UI terhadap suatu system. Pengujian API mirip dengan pengujian perangkat lunak di tingkat antarmuka pengguna, namun berbeda untuk pengujian dengan input dan output pengguna.

Cara kerja API *testing* adalah dengan menggunakan sebuah *software* untuk mengirim aksi dan kemudian software tersebut akan mencatat respons dari sistem. Kode pengujian dari API *test* akan mengirim aksi spesifik ke API kemudian akan mencatat respon sebenarnya dari system serta respon yang diharapkan oleh penguji.

Untuk otomasi API *testing* penguji biasanya menggunakan *tools* yang spesifik seperti contohnya IDE dengan menggunakan Eclipse dan bahasa Java. Tujuan utama penggunaan automasi adalah untuk mengotomatiskan pemeriksaan terhadap Aplikasi untuk mengetahui apakah aplikasi dibangun dan dikembangkan sesuai dengan requirement aplikasi tersebut.

3.8 Baseline Response time

Baseline merupakan penentuan suatu patokan ataupun tolak ukur. Pada penelitian ini, tolak ukur yang ditentukan ialah performansi *web service* pada *web application* pada Institut Teknologi Del. Penelitian ini akan menganalisis sebab dan akibat dari setiap proses penentuan *benchmark web service* pada *web application* yang ada di IT Del. Setelah penentuan *benchmark* performansi *web service* IT Del, maka Peneliti akan menjadikannya sebagai tolak ukur untuk dibandingkan suatu *web service* untuk dapat masuk ke dalam *server* IT Del.

Parameter *benchmark* yang ditentukan oleh peneliti adalah *response time web service* pada *website* ketika diberikan permintaan oleh pengguna. *Response time* adalah waktu yang diperlukan sistem untuk menanggapi permintaan dari pengguna. *Response time* dimulai dari pengguna mengirim permintaan dan berakhir saat aplikasi menyatakan bahwa permintaan tersebut telah selesai.

Dengan menganalisis *response time* dengan metode rata-rata saja, mungkin akan mendapatkan hasil yang berada pada tingkat yang dapat diterima, tetapi kurang akurat dikarenakan mungkin saja memiliki nilai individu data yang jauh dari nilai rata- rata. Perhitungan nilai rata-rata dapat dicari dengan menggunakan rumus matematis di bawah (dimana X adalah waktu respon):

$$X = \frac{X_1 + X_2 + \cdots + \cdots + X_n}{n}$$

Hal lainnya yang dapat menjadi metrik penentuan standar ialah Standart Deviasi. Standar deviasi merupakan ukuran penyebaran data terhadap rata-rata. Jika nilai standar deviasi kecil, maka menunjukkan bahwa semua nilai sampel mendekati rata-rata. Standar deviasi dapat meningkatkan keakuratan nilai yang akan menjadi benchmark melalui pemrosesan secara matematis. Perhitungan nilai standart deviasi dapat dicari dengan menggunakan rumus matematis di bawah (dimana X adalah waktu respon):.

$$s = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \underline{x})^2}{n-1}}$$

Nilai Rata-rata dan juga standar deviasi akan digunakan pada penelitian ini dalam menentukan *baseline* performansi *web service*.

3.9 *Response time*

Untuk menghitung lamanya waktu pemrosesan (response time) suatu API sampai API tersebut memberikan respon, peneliti melakukan perhitungan matematis terhadap faktor yang mempengaruhinya. Dibawah ini merupakan proses perhitungan manual response time pada pengujian provider telkomsel, tri, XL, dan jaringan Wifi. Berikut tahapan perhitungan matematis dalam memperoleh hasil Page Response time. Dalam menghitung response secara matematis menggunakan rumus di bawah ini (dimana X adalah waktu respon):

$$Download\ Time + DTturns + Processing\ Time\ Server + Processing\ Time\ Client$$

Untuk menghitung nilai Down*load* time menggunakan persamaan berikut :

$$\frac{Page\ Size}{Minimum\ Bandwidth}$$

Untuk menghitung nilai dari DT_{turns} :

$$Round\ Trip\ Time\ x\ \left(4 + \frac{3\ x\ NumObjects}{4}\right)$$

dengan bahasa pemrograman Java yang digunakan untuk menjalankan pengujian kinerja, pengujian beban dan pengujian fungsional aplikasi web. Aplikasi yang dibangun oleh Apache Software Foundation ini awalnya dirancang untuk menguji aplikasi website, menjadi diperluas ke fungsi pengujian lainnya. Apache JMeter dapat digunakan untuk menguji kinerja baik pada sumber daya statis dan dinamis, dan aplikasi dinamis web. Aplikasi ini digunakan untuk melakukan simulasi beban berat pada server, sekelompok server, jaringan atau objek untuk menguji kekuatannya atau untuk menganalisis kinerja keseluruhan di bawah jenis beban yang berbeda. JMeter merupakan aplikasi desktop yang berjalan di atas Java VM yang dapat mengukur kinerja sistem dan load- test dari aplikasi client ke server.

Meskipun JMeter dibangun menggunakan bahasa pemrograman Java, tetapi JMeter dapat menguji aplikasi web yang ditulis dengan banyak bahasa pemrograman seperti JSP/Servlet, PHP, ASP, Ruby, atau bahasa pemrograman lainnya (Nevedrov, 2006). JMeter juga dapat mensimulasikan beban berat pada server dengan membuat

banyak pengguna virtual bersamaan ke server web. Pada aplikasi JMeter, pengujian dapat dilakukan dengan menggunakan script atau tanpa menggunakan script. Pengujian tanpa menggunakan script dapat dilakukan dengan penyusunan sebuah TestPlan yang berisi satu atau lebih ThreadGroup dapat terdiri dari beberapa thread. Sebuah thread dapat dianalogikan sebagai seorang pengguna yang sedang mengakses aplikasi website.

IV. HASIL DAN PEMBAHASAN

4.1 *Load Testing*

Pada Sub bab ini akan dijelaskan mengenai hasil dan pembahasan mengenai *performance testing* yang telah dilakukan. Berikut merupakan data batas maksimum *user* pada *website* DiTenun yang dapat dilihat pada tabel di bawah ini.

Tabel 1 Jumlah Maksimum Use

No	Percobaan	Jumlah *User* Maksimum (Vu)
1	API Login	60 *User*
2	API Save Motif	1 *User*
3	API Save Motif Exited	60 *User*
4	API Motif *Load*	60 *User*
5	API Motif *Load* with Type	60 *User*
6	API Motif *Load* All	33 *User*
7	API Save Layout	60 *User*
8	API Save Layout Exited	60 *User*
9	API Get Layout	60 *User*
10	API Get Motif *Load* All	25 *User*

4.2 Apache Jmeter

JMeter merupakan alat pengujian yang digunakan untuk menganalisis dan mengukur kinerja berbagai layanan dan produk perangkat lunak. Tools ini merupakan tools yang berbasis *open source*. Berdasarkan data tabel 1 diatas maka direncanakan pengujian *Load Testing* pada API DiTenun dilakukan dengan beberapa kondisi, dimana jumlah pengguna aktif dibagi dalam beberapa kategori yaitu 10, 20, 30, 40, 50 pada API "Login", "Motif Save (Exited)", "Motif Load", "Motif load With Type", "Save Layout", "Save Layout (Exited)", dan "Get Layout" dan pada API "Motif Load All" kondisi yang dilakukan adalah dengan menggunakan virtual user sebesar 5, 10, 15, 20. Kategori jumlah pengguna ini ditentukan dibawah jumlah user maksimum, sehingga tingkat error yang didapatkan pada semua percobaan ialah 0% atau tidak terjadi. Berikut ini merupakan hasil pengujian dari pengujian Load Test pada API :

Tabel 2 Hasil Pengujian Load Test

API	Jumlah *request* (Vu)	Respon Time (ms)	*Throughput* (rps)	*Error* Rate
Login	10	1363	3,26	0%
	20	1654,6	5,76	0%
	30	2122,6	5,82	0%
	40	2470,6	7,3	0%
	50	2835,2	8,04	0%
Motif Save	1	462,8	2,32	0%
	2	684	2,74	0,50 %
	5	830,6	5,38	0,72 %
	10	856	9,18	0,88 %
Motif Save Exited	10	560,6	13,38	0%
	20	598	17,18	0%
	30	828,6	19,02	0%
	40	994	19,12	0%
	50	1206,4	19,38	0%
Motif *Load*	10	505,2	13,66	0%
	20	735,2	19,06	0%
	30	829,2	19,36	0%
	40	1238	19,56	0%
	50	1249,2	20,74	0%
Motif *Load* All With-Type	10	1321	3,58	0%
	20	1639,4	4,84	0%
	30	2241,8	5,74	0%
	40	2372,8	7,4	0%
	50	3214	7,52	0%
Motif Load All	5,00	19413,67	11,50	0%
	10,00	26177,67	14,80	0%
	15,00	36446,67	14,83	0%
	20,00	42773,00	16,07	0%
Save	10	513,4	14,8	0%

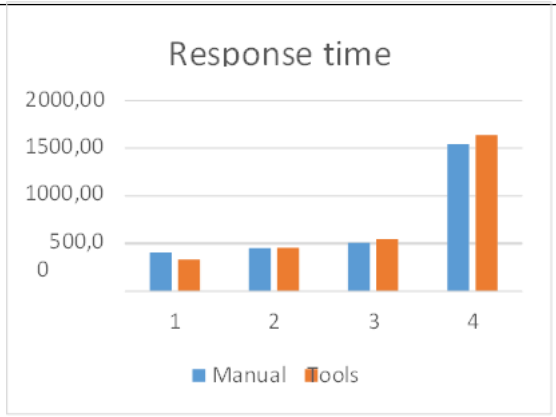
Layout	20	624	17,42	0%
	30	735,6	18,54	0%
	40	967,2	19,32	0%
	50	1283	19,5	0%
Save Layout Exited	10	550,6	13,62	0%
	20	576	18,6	0%
	30	742,2	19,38	0%
	40	920	19,5	0%
	50	1420,6	18,62	0%
Get Layout	10	636,4	12,72	0%
	20	803,6	15,64	0%
	30	977	15,86	0%
	40	1063	18,2	0%
	50	1134,2	20,56	0%

Berdasarkan hasil pengujian *Load Testing* setiap API yang dilakukan dengan JMeter, Setiap API memberikan perilaku yang sama dengan bentuk perilaku sebagai berikut:

- Dari hasil pengujian API secara *Load Test* dapat dilihat bahwa nilai dari avg.*response time* yang dihasilkan pada setiap API selalu mengalami peningkatan seiring bertambahnya jumlah *request* yang diberikan. Hal tersebut dapat membuktikan bahwa kinerja api akan semakin berkurang seiring dengan bertambahnya jumlah *user* yang mengaksesnya. Namun meskipun performa dari api mengalami penurunan, api tetap mampu mengatasi *request* dengan baik tanpa menimbulkan *error* atau kesalahan yang dapat mengganggu kenyamanan dari *user*.
- Perilaku *throughput* yang ditunjukan dalam pengujian *load* menggunakan JMeter cenderung mengalami peningkatan berdasarkan banyak *user* yang melakukan transaksi dengan *server*, nilai *throughput* yang dihasilkan dalam *tools* jmeter merupakan nilai dari jumlah *request* per waktu pengaksesan semua *request*.
- Hubungan antara *response time* dan nilai *throughput* yang dihasilkan adalah berbanding lurus dimana ketika nilai *response time* semakin meningkat maka nilai dari *throughput*nya akan semakin meningkat.
- Berdasarkan hasil pengujian yang dilakukan oleh penulis. Penulis mengidentifikasi *error* yang terdapat pada beberapa API ketika dilakukan Pengujian dengan menggunakan metode *Load Test* yaitu pada "API Motif Save" Adapun *error* yang didapati pada percobaan pengujian *Load Test* pada api Motif Save adalah masalah *Internal Serverrror* adalah dimana masalah tersebut sering dialami pada *server* sebuah *website*. Masalah tersebut timbul karena *web server* tidak mampu memenuhi apa yang diminta oleh pengunjung *website*.

4.3 *Baseline Response time*

Berdasarkan nilai *response time* yang didapat melalui pengukuran dengan menggunakan *tools* dan perhitungan matematis dapat dilihat dalam ilustrasi grafik di bawah ini :



Gambar 2 *Perbandingan Response time Tools dan Manual*

Berdasarkan grafik rata-rata response time perbandingan antara nilai response time berdasarkan perhitungan manual dan hasil pengukuran dengan menggunakan tools dapat dilihat tidak terdapat perbedaan yang signifikan antara 2 variabel yang dibandingkan. Perbedaan selisih tersebut kemungkinan terjadi akibat adanya fluktuasi jaringan yang dapat berubah-ubah sewaktu-waktu sehingga menyebabkan perbedaan response time.

Berdasarkan hasil pengukuran yang dilakukan maka dapat diambil kesimpulan baseline test yang dapat digunakan sebagai tolak ukur response time suatu API berdasarkan beban response yang diterima:

Tabel 3 Hasil Baseline Response time

Test case	Beban Response	Rata-Rata Pengukuran (ms)
1	594 B	333,7
2	905 B	458,41
3	1062 B	545,42
4	200000 B	1640,14

Berdasarkan tabel hasil rata-rata pengukuran diatas, dapat dilihat nilai rata- rata dari reponse time yang dihasilkan akan bertambah besar seiring dengan pertambahan nilai dari beban response yang dibuat. Berdasarkan data diatas dapat dilihat bahwa rata-rata *response time* dengan beban sebesar 598 B adalah 333,70 ms, pada beban sebesar 905 B adalah 458,41 ms, pada beban sebesar 1062 B adalah 545,42 ms, dan pada beban 200 KB adalah 1640,14 ms.

IV. KESIMPULAN

Dari hasil yang diperoleh pada penelitian ini, peneliti dapat menyimpulkan bahwa:

- Berdasarkan pengujian performansi dengan menggunakan metode *Load Test*, 8 dari 9 API mampu memberikan performansi yang baik dalam menerima *request* dan mengembalikan respon dimana meskipun API mengalami peningkatan *response time* yang mengindikasikan bahwa performa API mengalami penurunan berdasarkan jumlah *user* yang melakukan *request*, namun API tetap mampu menangani *request* tanpa menyebabkan *error* dan berdasarkan hasil pengujian dapat dilihat bahwa terdapat API yang mengalami permasalahan yakni "API Motif Save" dimana pada pada API "Motif Save" tidak dapat dilakukan pengujian performansi dengan

menggunakan metode *Load Test* karena terjadi *error* yang diakibatkan oleh *server* yang mengalami kondisi tak terduga dan melakukan pencegahan untuk memenuhi permintaan.

- Berdasarkan pengujian performansi dengan menggunakan metode *Load Test*, perilaku *throughput* yang ditunjukan dalam pengujian *load* menggunakan JMeter cenderung meningkat jumlahnya berdasarkan banyak *user* yang melakukan transaksi dengan *server*, hal ini menjadi indikator bahwa meskipun kapasitas *server* yang terus berkurang karena jumlah *request* dari *user* yang terus bertambah. Namun seluruh *request* tersebut berhasil diproses tanpa adanya gangguan dari *server*, yang ditandai dengan tidak adanya pesan *error* selama pengujian *Load Test* dengan jumlah *user* yang telah ditentukan.
- Pengukuran *baseline response time web* service telah dilakukan oleh Penulis. Berdasarkan pengukuran *baseline* yang sudah dilakukan, terdapat nilai rata-rata *response time* API *website* DiTenun dengan besar beban response sebesar 598 B adalah 333,70 ms, pada beban sebesar 905 B adalah 458,41 ms, pada beban sebesar 1062 B adalah 545,42 ms, dan pada beban 200 KB adalah 1640,14 ms.

UCAPAN TERIMAKASIH

Kami mengucapkan terima kasih kepada LPDP yang sudah memberikan dukungan dalam pelaksanaan penelitian ini dalam skema Rispro Komersial dengan judul Piranti Cerdas Penghasil Tenun Nusantara

REFERENCES

Aditya, Dony, dkk. 2020. E-Commerce Marketplace Petshop Menggunakan Integrasi Rajaongkir API dan iPaymu Payment Gateway API. Bali:Jurnal Jitter.

Afijal. 2018.Teknik Pengujian Perangkat Lunak.Jakarta:Rekayasa Perangkat Lunak.

Busran, & Ridwan, Ahmad. 2020. Analisis Perbandingan Performa Apache *Web server* dan Nginx Menggunakan Apache Jmeter. Padang:Jurnal Teknoif.

Firdaus, Ali, dkk. 2019. Rancang Bangun Sistem Informasi Perpustakaan Menggunakan *Web* Service Pada Jurusan Teknikkomputer Polsri.Palembang :Jurnal Informatika.

Gunawan Hendro, dkk. 2017. Pemanfataan Aplikasi Mobile Untuk Mempercepat Pencarian Tempat Indekos Berbasis Android.Yogyakarta : Jurnal Jurnal Muara Sains.

Hasnain, Muhammad, dkk. 2020.An Efficient *Performance testing* of *Web* Services. Rawalpindi: IEEE.

Jha, Nisha, & Popli, Rashmi. 2019. Comparative Analysis of *Web* Applications using JMeter.Faridabad : International Journal of Advanced Research in Computer Science.

Kurniawan, Erick. 2014. Implementasi Rest *Web* Service Untuk Sales Order Dan Sales Tracking Berbasis Mobile. Bali: Jurnal Eksis.

Liu, Feming, at all. 2014. A *Web* Service trust evaluation model *based* on small-world networks. Ji'nan: Knowledge-*Based* Systems.

Mukharil, dkk. 2013. Analisis Kualitas Perangkat Lunak Terhadap Sistem Informasi Unikom. Bandung: Majalah Ilmiah UNIKOM.

Nofia, Evin.2016.*Performance test* Dan Stress *Website* Menggunakan *Open source Tools*.Surabaya : Jurnal Manajemen Informatika.

Nurudin, Muhamad, dkk. 2019. Pengujian Black Box pada Aplikasi Penjualan Berbasis *Web* Menggunakan Teknik Boundary ValueAnalysis. Banten: Jurnal Informatika Universitas Pamulang.

Savoia, Alberto. 2001. *Web Response time* 101. United States: STQE.

Sha, Fei, at all. 2010. A Performance Evaluation Method And It's Implementation For *Web* Service.Beijing: Proceedings of IC-BNM.

Yohanes, Andre, dkk.2019.Pengaruh Pertambahan Jumlah *User* terhadap Performansi Pengalokasian Sumber Daya Radio Pada Sistem MIMO-OFDMA 2x2 Berbasis Quality of Service (QoS) Guaranteed. Bandung: Telkom